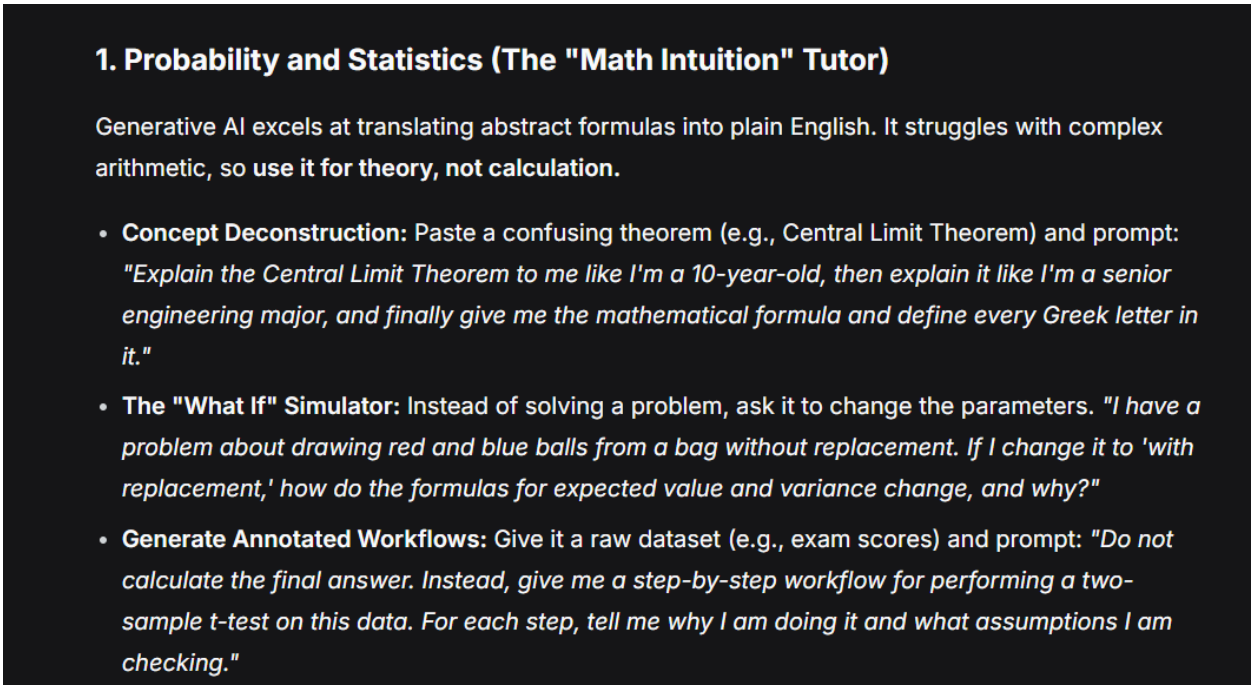
Exercise 3

Prompt:

How you can use generative AI as a university student to learn subjects like Probability and Statistics, Database Management Systems and Software Engineering

1) deepseek



1. Probability and Statistics (The "Math Intuition" Tutor)

Generative AI excels at translating abstract formulas into plain English. It struggles with complex arithmetic, so **use it for theory, not calculation.**

- **Concept Deconstruction:** Paste a confusing theorem (e.g., Central Limit Theorem) and prompt: *"Explain the Central Limit Theorem to me like I'm a 10-year-old, then explain it like I'm a senior engineering major, and finally give me the mathematical formula and define every Greek letter in it."*
- **The "What If" Simulator:** Instead of solving a problem, ask it to change the parameters. *"I have a problem about drawing red and blue balls from a bag without replacement. If I change it to 'with replacement,' how do the formulas for expected value and variance change, and why?"*
- **Generate Annotated Workflows:** Give it a raw dataset (e.g., exam scores) and prompt: *"Do not calculate the final answer. Instead, give me a step-by-step workflow for performing a two-sample t-test on this data. For each step, tell me why I am doing it and what assumptions I am checking."*

2. Database Management Systems (The "SQL Syntax & Design" Coach)

DBMS is highly logical and syntax-heavy. AI is a brilliant pair-programmer for SQL, but a terrible data modeler. **Use it for syntax and normalization, but always sketch designs on paper first.**

- **Rubber-Duck Debugging:** Paste your broken SQL query and prompt: *"I am getting a 'column ambiguously defined' error. Don't rewrite the query. Instead, act as a debugger and ask me three leading questions that will help me find the error myself."*
- **Normalization Practice:** Give it a messy, denormalized table (e.g., *OrderID, CustomerName, ProductID, ProductPrice, Quantity*) and prompt: *"Walk me through the process of normalizing this from 1NF to 3NF. For each step, explain which functional dependency you are removing and why. Show the final ER diagram in text format."*
- **Indexing & Performance Scenarios:** Prompt: *"I have a query that joins a 'Users' table (1M rows) and 'Orders' table (5M rows) on UserID. Give me a scenario-based question: Under what conditions would a full table scan be faster than using a B-tree index? Explain the concept of cardinality here."*

3. Software Engineering (The "System Design & Process" Mentor)

SE is about trade-offs, architecture, and process—not just code. AI is fantastic for simulating design discussions and generating project scaffolding.

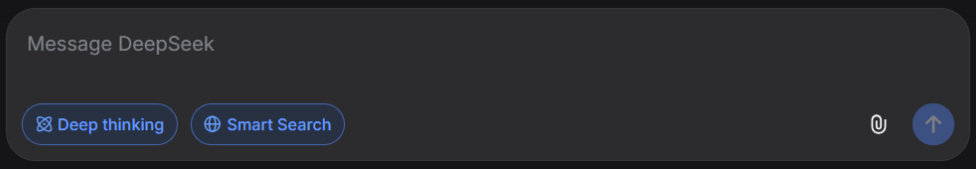
- **Design Review Simulator:** Prompt: *"Act as a senior software architect. I am designing a ride-sharing app. My current idea is to use a monolithic REST API with a PostgreSQL database. Challenge my design. Give me three scenarios where this fails, and suggest alternatives, but have me choose which trade-off to accept."*
- **Agile/Scrum Story Splitting:** Prompt: *"Here is a user story: 'As a user, I want to reset my password via email.' Break this down into 5 smaller technical tasks for a sprint. For each task, list the acceptance criteria and which layer (Frontend, Backend, Database, Email Service) it touches."*
- **UML & Diagramming Assistance:** Prompt: *"Generate a PlantUML or Mermaid code for a class diagram representing a Library Management System. Include the 'Library,' 'Book,' 'Member,' and 'Loan' classes with their attributes and methods. Do not include relationships yet—I will add those myself based on your explanation of multiplicity."* (This gives you a head start on visuals while you focus on the logic).

Universal "Golden Rules" for Using AI as a Student

To avoid the trap of "cheating yourself," adopt these three rigid habits for all subjects:

1. **The "No Final Answer" Rule:** Prepend every prompt with: *"Do not give me the final answer. Guide me to it."* If the AI gives you the answer anyway, reply: *"Hide the final answer. Give me a hint and a checkpoint to verify my work."*
2. **The "Feynman Technique":** After the AI explains a concept, prompt it: *"Now, quiz me on that explanation. Ask me one question at a time. If I get it right, move to the next. If I get it wrong, explain the misconception without giving me the new answer."*
3. **The "Citation Requirement":** For SE and DBMS, prompt: *"When giving me best practices (e.g., indexing strategies or design patterns), cite the source textbook or industry standard (e.g., Martin Fowler, or the PostgreSQL manual) so I can go read the original material for deeper understanding."*

Finally, **use the "Custom Instructions"** feature in your AI tool. Tell it: *"I am a 2nd-year university student. I am using you to learn. Prioritize Socratic questioning over providing solutions. Always ask me 'Why?' before you give an explanation."* This will completely change the quality of your interactions.



2) Qwen

🎲 1. Probability and Statistics

This subject is highly conceptual and math-heavy. GenAI excels at breaking down complex formulas and providing endless practice.

- **Socratic Problem Solving**: Instead of asking for the answer, prompt: *"I'm trying to solve this probability problem: [insert problem]. Don't give me the answer. Instead, ask me guiding questions one at a time to help me figure out the next step."*
- **Conceptual Analogies**: Ask for real-world analogies. *"Explain the Central Limit Theorem to me using an analogy about baking cookies."*
- **Generate Custom Practice Problems**: *"Generate 3 practice problems on Bayes' Theorem: one easy, one medium, and one hard. Provide the answers hidden under a spoiler tag or at the very end."*
- **Leverage Code Interpreter**: Use AI with code execution capabilities (like Python) to simulate statistical concepts. Prompt: *"Write a Python script to simulate the Monty Hall problem 10,000 times and plot the win rates for switching vs. staying to empirically prove the probability."*
- **Debugging Math Steps**: If you get a wrong answer, paste your step-by-step work and ask: *"Here is my derivation for this variance problem. At which step did I make a logical or arithmetic error?"*

🗄️ 2. Database Management Systems (DBMS)

DBMS combines theoretical concepts (normalization, ACID properties) with practical skills (SQL, schema design). GenAI acts as an excellent peer programmer and database architect.

- **Interactive Normalization Practice**: Prompt: *"Give me an unnormalized dataset representing a university course enrollment system. Ask me to convert it to 1NF, 2NF, and 3NF step-by-step, and grade my responses."*
- **SQL Query Debugging & Optimization**: Paste your broken SQL query and ask: *"Why is this query returning a Cartesian product? Also, how can I optimize it using indexes or a CTE?"*
- **Schema Design Feedback**: Describe your project idea and ask: *"I am designing a database for a library management system. Here are my tables and relationships. Critique my design for potential anomalies and suggest improvements."*
- **Concept Visualization**: Ask the AI to generate Mermaid.js or PlantUML code for Entity-Relationship (ER) diagrams. You can then paste this code into a free online viewer to instantly visualize your database schema.
- **Explain Execution Plans**: Paste a database execution plan and ask: *"Explain this query execution plan in simple terms. Why is it doing a full table scan instead of using an index?"*

🪜 3. Software Engineering

Software Engineering is about processes, architecture, design patterns, and teamwork, not just coding. GenAI can simulate real-world industry scenarios.

- **Design Pattern Tutor**: *"Explain the Observer Design Pattern. Show me a practical example in Java/Python, and then tell me a scenario where using this pattern would be a bad idea."*
- **Code Reviews for SOLID Principles**: Paste your code and prompt: *"Act as a senior software engineer. Review this code specifically for violations of the SOLID principles and suggest refactored versions."*
- **Simulate Stakeholder Interviews**: Prompt: *"Act as a non-technical client who wants a food delivery app. I will ask you questions to gather requirements. Respond realistically, and then help me draft User Stories and Acceptance Criteria based on our conversation."*
- **Generate Boilerplate & Documentation**: Use AI to speed up tedious tasks. *"Generate a standard README.md template for a Python Flask project, including sections for installation, API endpoints, and testing."*
- **Testing Assistance**: *"Here is my Python function. Generate comprehensive unit tests using `pytest`, including edge cases and mock objects for external API calls."*

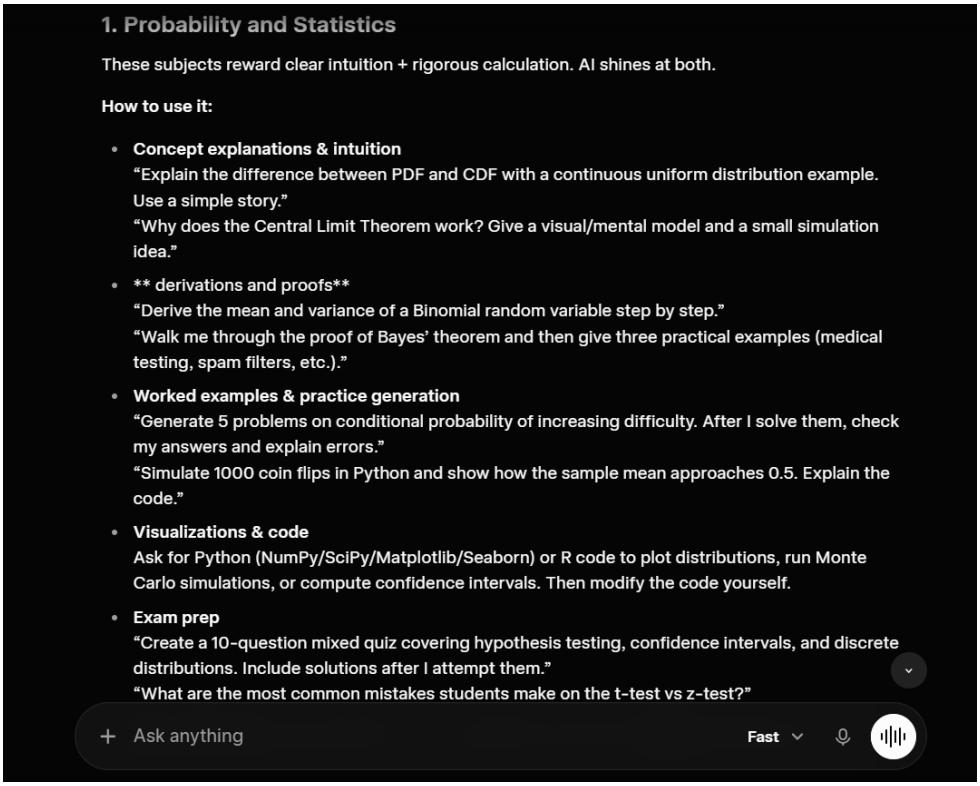
🧠 Cross-Cutting Best Practices for University Students

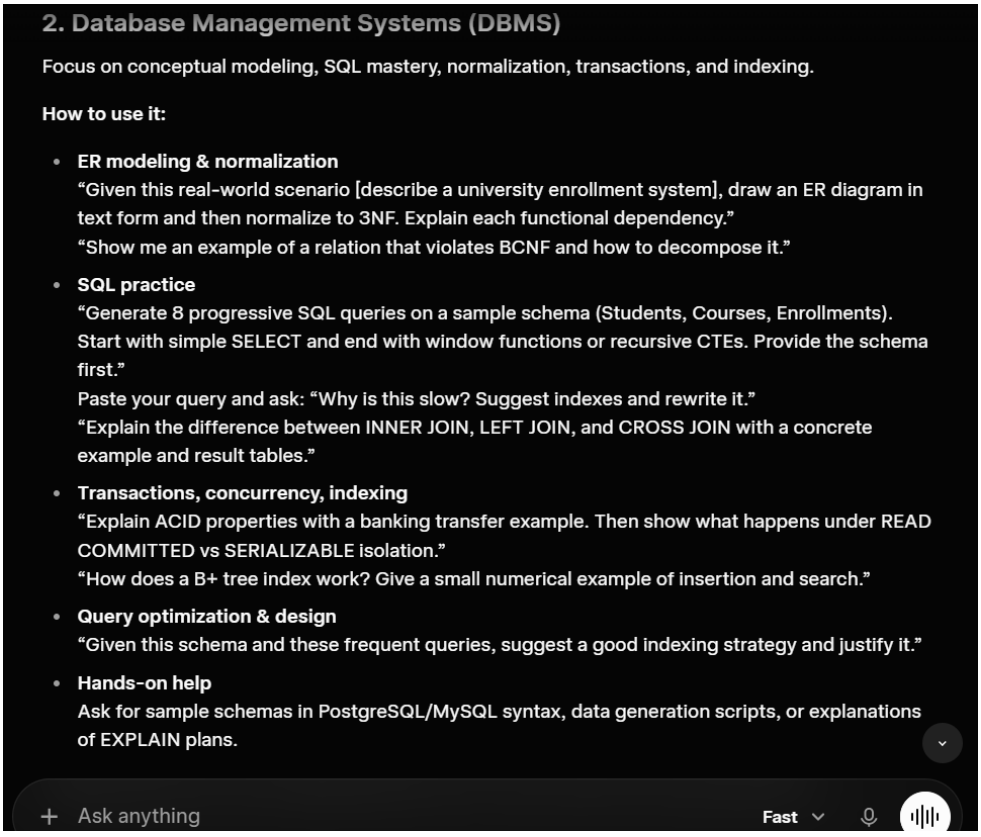
1. **Use the "Feynman Technique" Prompt**: *"I just learned about [Concept]. I will explain it to you in simple terms. Tell me where my explanation is inaccurate or incomplete."*
2. **Beware of Hallucinations**: Generative AI can confidently state false facts, especially with obscure statistical formulas or deprecated SQL syntax. **Always verify** critical information with your textbook, lecture notes, or official documentation.
3. **Maintain Academic Integrity**: Use AI as a *tutor*, not a *ghostwriter*. Do not paste take-home exam questions or submit AI-generated code/essays as your own work. Your goal is to build neural pathways in your own brain, not the AI's.
4. **Provide Context**: The better your prompt, the better the output. Instead of *"Explain joins,"* say *"I understand INNER JOIN, but I'm confused about the practical difference between LEFT JOIN and FULL OUTER JOIN. Explain with a student-course enrollment example."*
5. **Create Study Aids**: Prompt: *"Turn these lecture notes on [topic] into a set of 10 Anki-style flashcards (Question and Answer format) focusing on the most testable concepts."*

🛠️ Recommended AI Workflow

1. **Learn**: Use AI to get a high-level overview of a new topic before lecture.
2. **Practice**: Use AI to generate custom problems and grade your attempts.
3. **Debug**: Use AI (especially with Code Interpreter) to find errors in your SQL or code.
4. **Review**: Use AI to quiz you or summarize your notes before an exam.

3) Grok


1. Probability and Statistics
These subjects reward clear intuition + rigorous calculation. AI shines at both.
How to use it:
Concept explanations & intuition
“Explain the difference between PDF and CDF with a continuous uniform distribution example. Use a simple story.”
“Why does the Central Limit Theorem work? Give a visual/mental model and a small simulation idea.”
** derivations and proofs**
“Derive the mean and variance of a Binomial random variable step by step.”
“Walk me through the proof of Bayes’ theorem and then give three practical examples (medical testing, spam filters, etc.).”
Worked examples & practice generation
“Generate 5 problems on conditional probability of increasing difficulty. After I solve them, check my answers and explain errors.”
“Simulate 1000 coin flips in Python and show how the sample mean approaches 0.5. Explain the code.”
Visualizations & code
Ask for Python (NumPy/SciPy/Matplotlib/Seaborn) or R code to plot distributions, run Monte Carlo simulations, or compute confidence intervals. Then modify the code yourself.
Exam prep
“Create a 10-question mixed quiz covering hypothesis testing, confidence intervals, and discrete distributions. Include solutions after I attempt them.”
“What are the most common mistakes students make on the t-test vs z-test?”
Ask anything
Fast


2. Database Management Systems (DBMS)
Focus on conceptual modeling, SQL mastery, normalization, transactions, and indexing.
How to use it:
ER modeling & normalization
“Given this real-world scenario [describe a university enrollment system], draw an ER diagram in text form and then normalize to 3NF. Explain each functional dependency.”
“Show me an example of a relation that violates BCNF and how to decompose it.”
SQL practice
“Generate 8 progressive SQL queries on a sample schema (Students, Courses, Enrollments). Start with simple SELECT and end with window functions or recursive CTEs. Provide the schema first.”
Paste your query and ask: “Why is this slow? Suggest indexes and rewrite it.”
“Explain the difference between INNER JOIN, LEFT JOIN, and CROSS JOIN with a concrete example and result tables.”
Transactions, concurrency, indexing
“Explain ACID properties with a banking transfer example. Then show what happens under READ COMMITTED vs SERIALIZABLE isolation.”
“How does a B+ tree index work? Give a small numerical example of insertion and search.”
Query optimization & design
“Given this schema and these frequent queries, suggest a good indexing strategy and justify it.”
Hands-on help
Ask for sample schemas in PostgreSQL/MySQL syntax, data generation scripts, or explanations of EXPLAIN plans.
Ask anything
Fast

3. Software Engineering

Covers requirements, design patterns, architecture, testing, version control, Agile, and code quality.

How to use it:

- **Requirements & design**
  "Help me write a Software Requirements Specification outline for a library management system. Include functional and non-functional requirements."
  "Given these user stories, suggest a high-level architecture (layered, microservices, etc.) and justify the choice."
- **Design patterns & UML**
  "Explain the Observer pattern with a real example (e.g., stock price updates). Show UML class diagram in text and sample code in Java/Python."
  "When should I use Strategy vs State vs Template Method? Give decision criteria."
- **Code quality, refactoring, testing**
  Paste a messy function and ask: "Refactor this for readability and SOLID principles. Explain every change."
  "Write unit tests (pytest/JUnit) for this class. Cover edge cases."
  "Generate a sequence diagram for the login flow of a web app."
- **Process & tools**
  "Explain the differences between Waterfall, Agile, and DevOps in the context of a student project. Recommend a lightweight process for a 4-person team."
  "Help me write a good Git commit message and a pull-request description for this change."
- **Project help**
  Use it for brainstorming architectures, generating boilerplate (with understanding), reviewing your design documents, or simulating code reviews.

Practical Workflow Recommendations

1. **After every lecture** → Summarize key points yourself, then ask AI to expand, correct, or give analogies.
2. **Before solving problems** → Attempt first, then ask for hints or full solutions only after struggling.
3. **Code-heavy subjects** → Generate + modify + run the code yourself. Ask "Why does this line work?" often.
4. **Group study** → Use AI to generate discussion questions or alternative solutions to compare.
5. **Long-term retention** → Ask AI to create spaced-repetition style flashcards or "explain it back to me as if I'm teaching a junior."

Limitations & Good Habits

- AI can be wrong on edge cases, recent standards, or complex proofs—always cross-check.
- For graded work, follow your university's academic integrity policy strictly. Using AI for understanding is usually fine; submitting its output as your own is not.
- Build the habit of prompting well: be specific, give context, request step-by-step reasoning, and ask for sources or verification methods when possible.

Used this way, generative AI becomes a personalized, always-available tutor that helps you move from "I can follow the lecture" to "I can teach this topic and solve novel problems." Start with one subject this week—pick a concept that confused you and ask for three different explanations plus practice problems.